

AI Blog Nest API

Project Title: [AI Blog Nest API – An Intelligent Backend for Blog Operations](#)

Team ID : SWTID-2026-4636

Team Size : 4

Team Leader : SAI TEJA SHRI P

Team member :

SAI TEJA SHRI P

YAMINISREE P

HARSHITHA K

ISHWARYALAKSHMI S

1. Project Overview

AI Blog Nest API is a secure and intelligent REST API platform designed to simplify blog creation, content management, and AI-assisted writing.

The backend is built using Node.js and Express.js and uses MongoDB with Mongoose ODM for efficient data storage, validation, and database management.

The platform integrates Google Gemini AI for AI-powered blog generation and content summarization. It also supports user roles, blog management, categories and tags, comments, media metadata, search, and centralized error handling.

Security features include JWT authentication, role-based access control for Admin, Editor, Author, and Reader roles, bcrypt.js password hashing, request validation, input sanitization, rate limiting, and structured logging.

Project Instructions – AI Blog Nest API

- Register and authenticate a user through the API.
- Use JWT-based authentication to access protected endpoints.
- Create, view, update, and manage blog posts according to the user's role.
- Generate blog content using the AI generation endpoint.
- Summarize existing blog content using the AI summarization endpoint.
- Organize posts using categories and tags.
- Manage and moderate comments through protected routes.
- Use Postman or ThunderClient to test API endpoints.

Problems

- Difficulty managing multiple drafts, categories, and tags.
- Limited visibility into post performance and engagement analytics.
- Time-consuming content approval and comment moderation.
- No streamlined system for media uploads or metadata.
- Lack of collaborative permission tiers for editors, authors, and administrators.

2. Pre-requisites

Basic Knowledge.

- Node.js and Express.js backend development.
- RESTful API concepts and HTTP methods.
- MongoDB and basic CRUD operations.
- Authentication and authorization concepts.
- Environment variable configuration.

Software Requirements

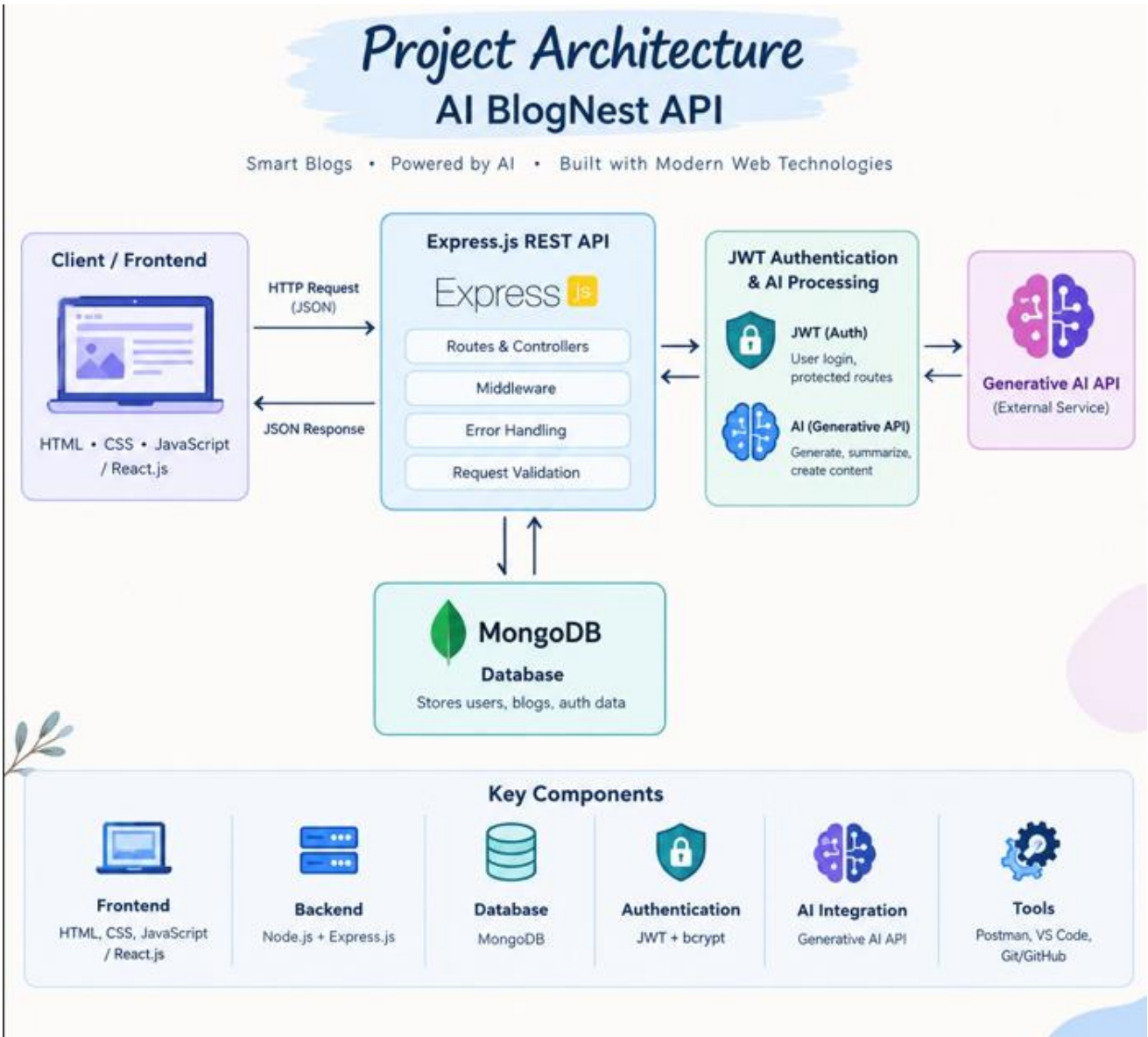
- Operating System: Windows 10/11
- Node.js version 16 or above.
- npm version 8 or above.
- Express.js framework.
- MongoDB database.
- Mongoose ODM.
- Postman for API testing.
- Visual Studio Code or another code editor.

Additional Understanding

- JWT authentication and role-based access control.
- Password hashing using bcrypt.js.
- JSON request and response formats.
- AI service integration and prompt-based content generation.

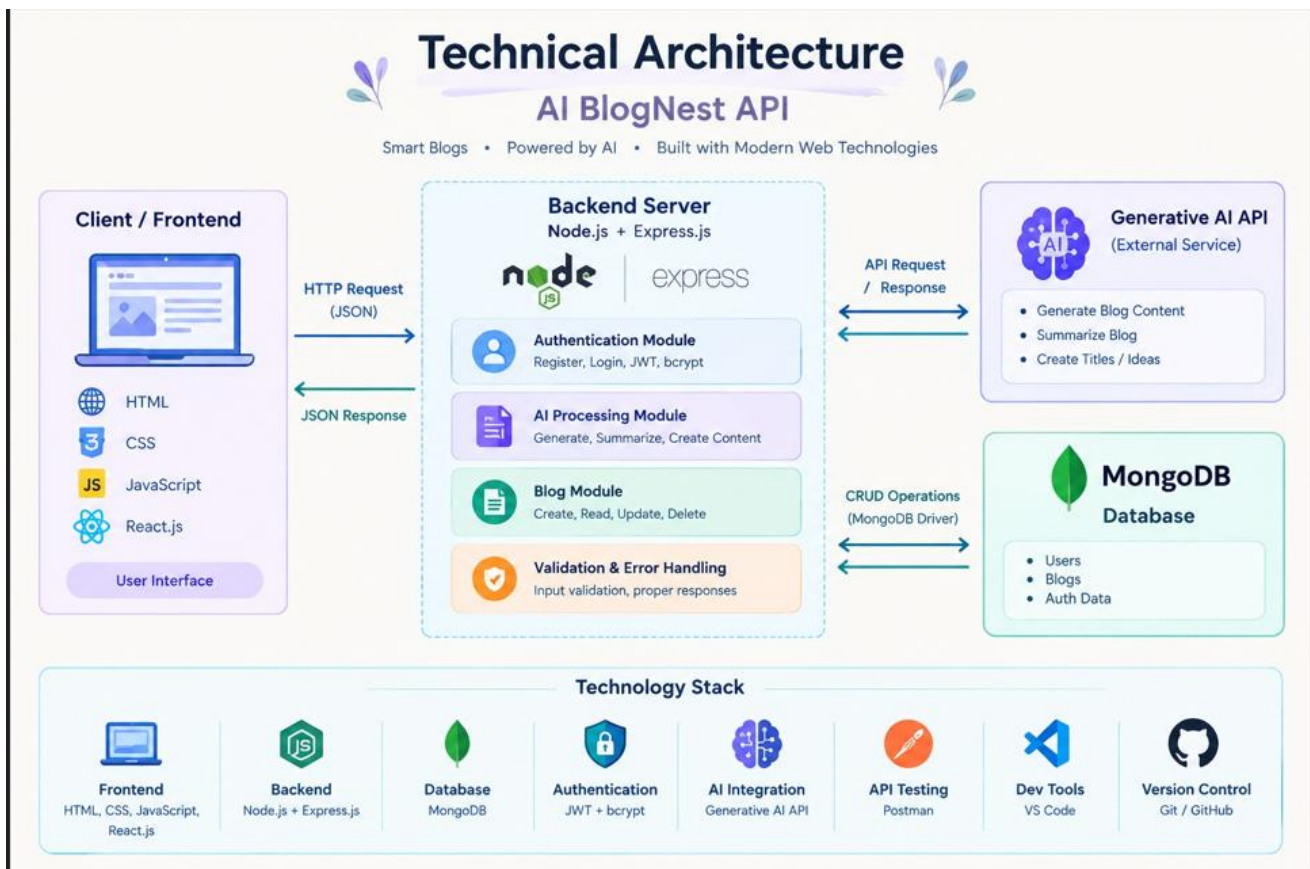
Epic:-1. Project Architecture

AI Blog Nest follows a layered server architecture. Incoming requests pass through authentication and validation middleware before reaching business services and database operations.



Technical Architecture

The architecture is organized to keep routing, business logic, AI processing, and data access separate. This improves maintainability, scalability, security, and easier debugging.



1. API Layer – Routes

- Defines RESTful API endpoints.
- Handles GET, POST, PUT, and DELETE methods.
- Maps incoming requests to controllers or services.
- Applies authentication and validation middleware.

2. Business Logic Layer – Controllers / Services

- Processes blog creation and updates.
- Handles publishing states such as Draft, Pending Approval, Scheduled, and Published.
- Manages comments, categories, tags, and user permissions.
- Calls AI services when content generation or summarization is requested.

3. Data Access Layer – Models / Database

- Uses Mongoose schemas and models.
- Stores and retrieves users, posts, and related records.
- Validates data before persistence.
- Maintains logical relationships between users, posts, and comments.

4. AI Flow – Service Integration

The AI Flow provides intelligent content automation. Specialized endpoints such as `/api/ai/generate-blog` and `/api/ai/summarize` allow the controller layer to send text-processing tasks to the integrated AI service.

Key Features

User Authentication and Security

- JWT-based authentication for protected resources.
- Role-based access control for Admin, Editor, Author, and Reader.
- Secure password hashing using bcrypt.js.
- Request validation and input sanitization.
- Rate limiting and structured logging.
- Centralized error handling.
- Protection against unauthorized access.

Blog Management – CRUD Operations

- Create blog posts.
- Retrieve blog posts.
- Update blog details.
- Delete blog posts.
- Manage post lifecycle and publishing status.
- Assign categories, tags, and media metadata.

AI-Powered Features

- Generate complete blog content from high-level thematic inputs.
- Summarize extensive article content into concise abstracts.
- Use AI-assisted writing to reduce manual drafting effort.
- Integrate AI processing through dedicated REST API endpoints.

Comment and Content Management

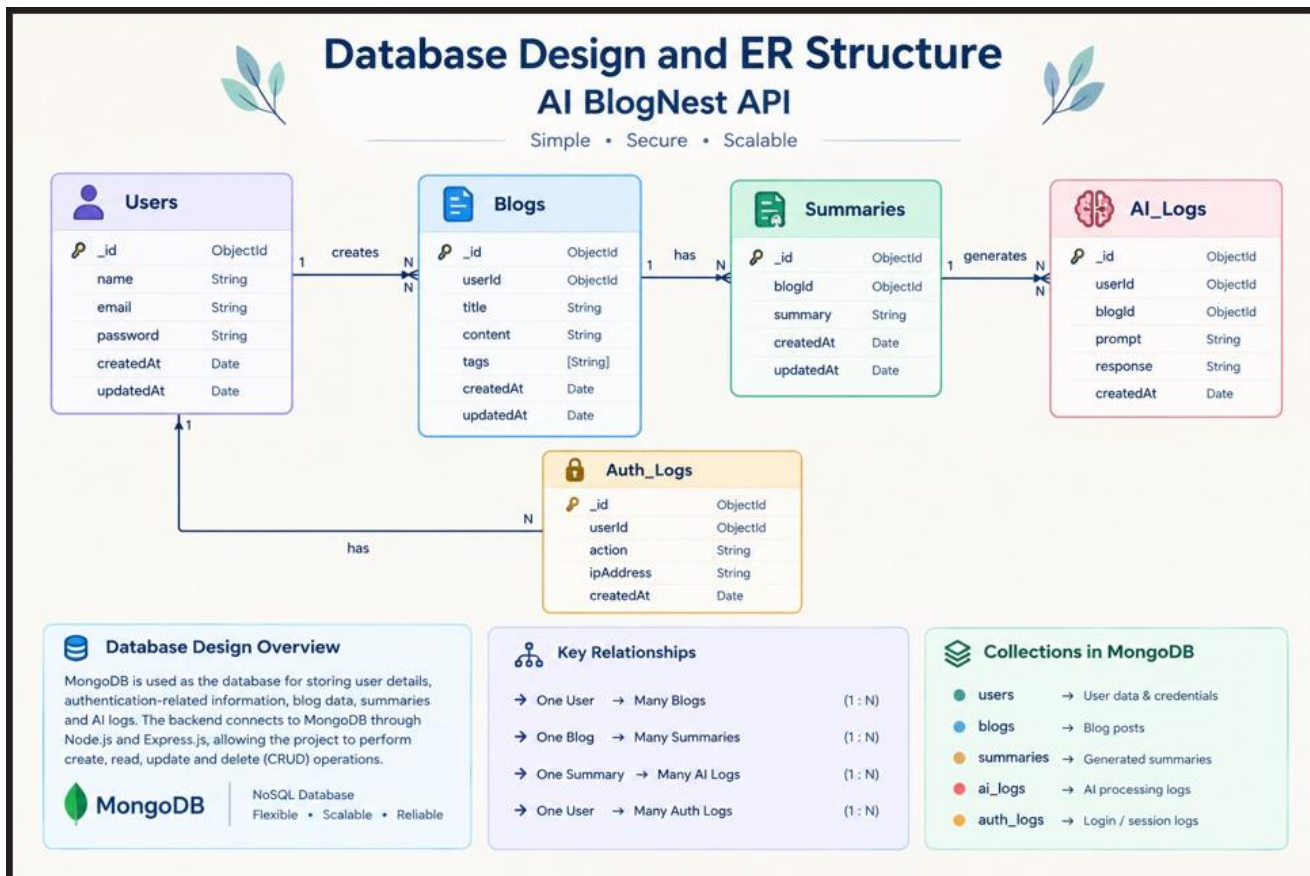
- Create and manage comments.
- Moderate comments using protected endpoints.
- Support moderation states such as approved, pending, and spam.
- Allow Editors/Admins to manage content workflows.

Categorization and Search

- Organize content with categories and tags.
- Support filtering and discoverability through tag indices.
- Provide search-oriented backend operations.

Database Design and ER Structure

The BlogNest data model uses MongoDB collections connected through logical application references.



The BlogNest data model uses MongoDB collections connected through logical application references.

User Entity

- `_id` – Primary Key (ObjectId)
- `name` – String, Required
- `email` – String, Required and Unique
- `password` – String, Hashed and Required
- `role` – String, Default: reader

Post Entity

- `_id` – Primary Key (ObjectId)
- `userID` – Foreign Key reference to User Collection
- `name` – String, Required; represents the blog title
- `photo` – Object/String pathway link
- `message` – String; core blog content
- `likes` – Number, Default: 0
- `timestamps` – Enabled

Key Structural Mappings

Relationship Description

User → Post One user/author can submit multiple blog documents.

User → Comment A user can create multiple comments.

Post → Comment A single blog post can contain multiple comments.

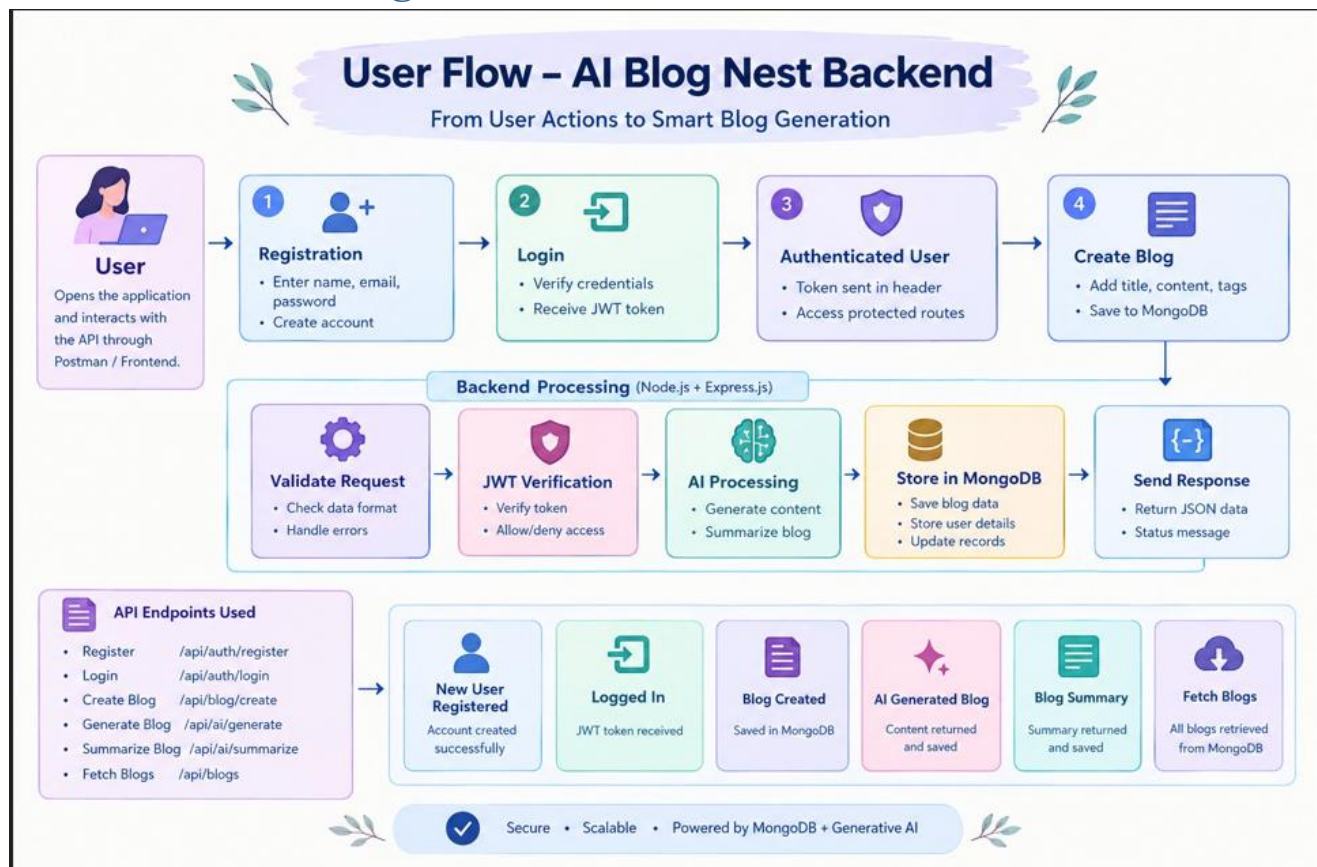
Data Validation and Protection

- Mongoose schemas define required fields and validation rules.
- Passwords are stored in hashed form.
- Protected routes verify JWT authentication.
- Sanitization reduces unsafe input and NoSQL/XSS-related risks.
- Role checks prevent unauthorized modification of protected resources.

Roles and Responsibilities

Project Team Roles Role Responsibilities Backend Developer Develop Node.js and Express.js APIs Database Developer Design and manage MongoDB collections AI Integration Developer Integrate AI for blog generation and summarization API Tester Test APIs using Postman and verify responses Documentation Prepare project documentation and screenshots Key Responsibilities Develop and maintain REST APIs. Implement JWT authentication and bcrypt password security. Connect the application with MongoDB. Integrate Generative AI features. Perform API testing using Postman. Handle validation and errors. Maintain project documentation.

User Flow – AI Blog Nest Backend



User Registration

1. User sends a registration request.
2. System validates required user fields.
3. Password is securely hashed.
4. User record is stored in MongoDB.
5. Success response is returned.

User Login

6. User submits email and password.
7. System verifies credentials.
8. A JWT token is generated for a valid login.
9. Token is returned to the client.
10. User can now access protected resources according to their role.

Access Protected Routes

Authorization: Bearer <token>

↓

JWT verification

↓

Role / permission check

↓

Controller / Service

↓

JSON response

Blog Operations

- Create Blog → POST /api/blog
- Fetch Blogs → GET /api/blog
- Update Blog → PUT /api/blog/:id
- Delete Blog → DELETE /api/blog/:id

AI Operations

- Generate AI Blog → POST /api/ai/generate-blog
- Summarize Blog → POST /api/ai/summarize

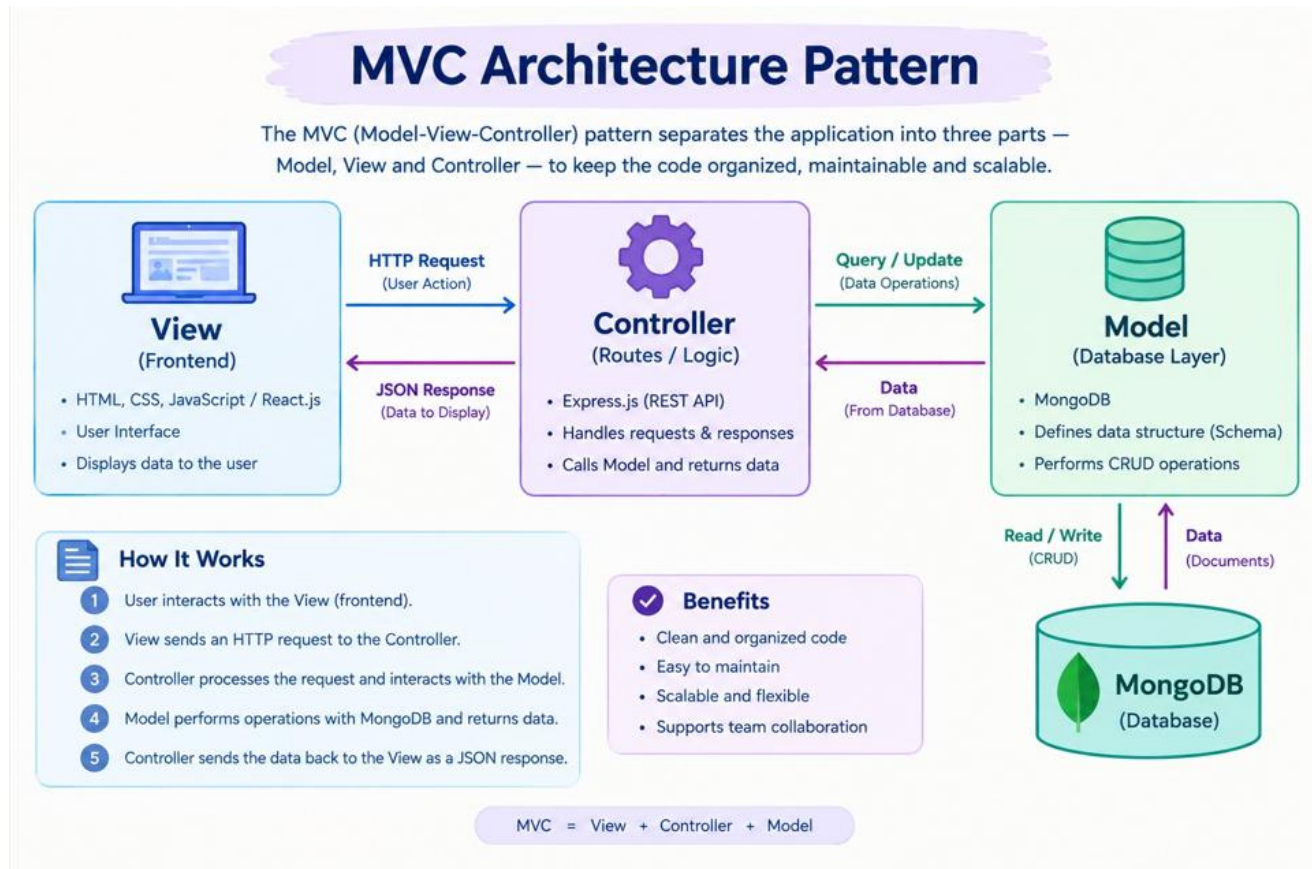
Overall Flow Summary

Register → Login → Receive JWT → Access Protected Routes → Create / Manage Blog → Generate or Summarize with AI → Moderate / Publish → Read / Comment

MVC Architecture Pattern

The MVC (Model-View-Controller) pattern separates the application into three parts: Model, View, and Controller.

It keeps the AI BlogNest API organized, maintainable, and easy to manage.

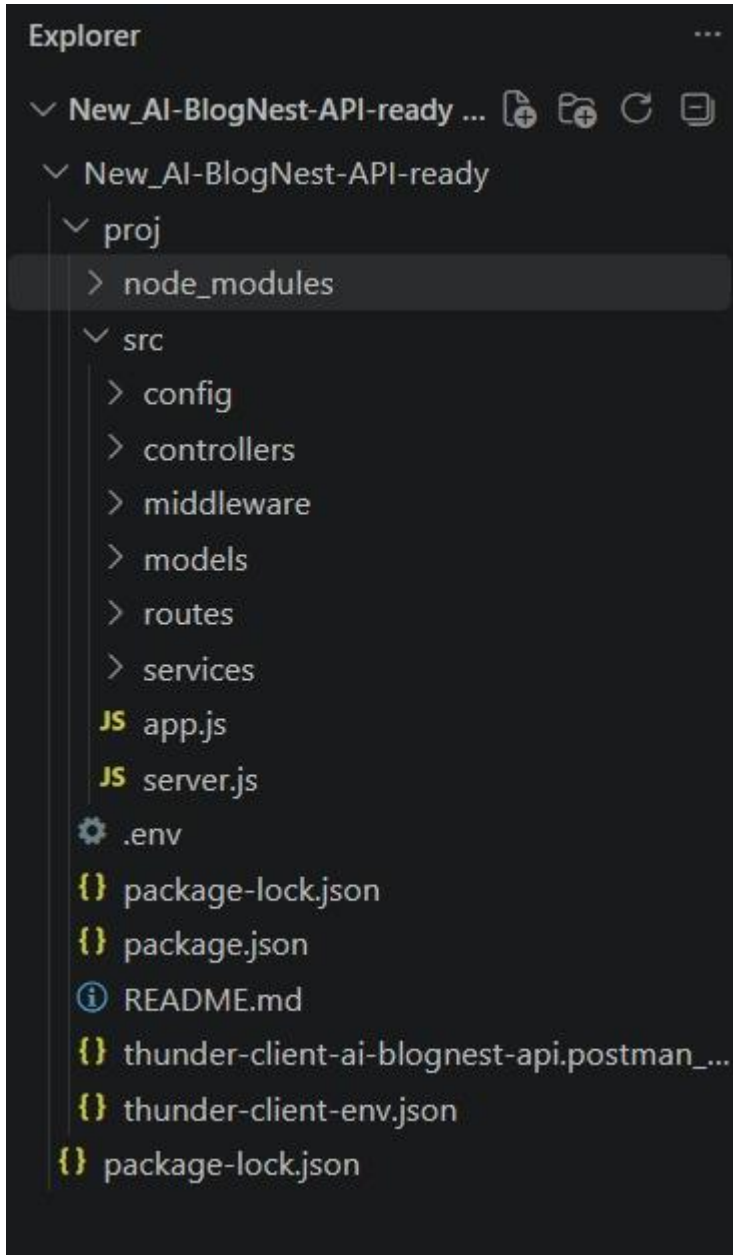


Epic:-2 Project Setup and Configuration

Creating Project Folder

1. Create a new folder and name it AI BlogNest API.
2. Inside the project folder, create a new folder named Server.
3. Open the Server folder in Visual Studio Code.
4. The project is now ready for backend development.

Folder Structure



Initialize Node.js Project

```
npm init -y
```

Install Dependencies

1. Open the VS Code Terminal inside the project folder.
2. Use `dir` to view the project files.
3. Use `cd proj` to enter the project folder.
4. Use `dir` again to verify the project files.
5. Install the required packages using `npm install express mongoose bcryptjs jsonwebtoken dotenv cors`.
6. Run `npm audit fix` to fix dependency-related issues.
7. Install Nodemon using `npm install --save-dev nodemon`.

Environment Variables

PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_gemini_api_key

Epic 3: Backend Development

The backend is developed using Node.js and Express.js following RESTful architecture principles. It handles authentication, business logic, database operations, AI service integration, and API responses.

Backend Development Server.js

```
New_AI-BlogNest-API-ready > proj > src > JS server.js > ...
1  const dns = require('dns');
2
3  dns.setServers(['8.8.8.8', '1.1.1.1']);
4
5  const dotenv = require('dotenv');
6  dotenv.config();
7
8  const app = require('./app');
9  const connectDB = require('./config/db');
10
11 connectDB();
12
13 const PORT = process.env.PORT || 5000;
14
15 app.listen(PORT, () => {
16   console.log(`Server running on port ${PORT}`);
17 });
18
```

App.js

```

New_AI-BlogNest-API-ready > proj > src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22    | res.json({ message: 'AI BlogNest API is running' });
23    | });
24
25  module.exports = app;

```

Epic 4: Database Configuration :-

DB Connection

```

New_AI-BlogNest-API-ready > proj > src > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;

```

1. Configure MongoDB

- Install Mongoose to connect the Node.js application with MongoDB.
- In your Node.js project directory, run: `npm install mongoose`
- Create the MongoDB database connection.
- Create the required Schemas and Models for users and blogs.

Create Database Connection

Make sure the MongoDB database is connected before performing any backend operations. The connection code is added to the Node.js project to establish a connection between the Express.js server and MongoDB.

Mongoose Schema Definitions

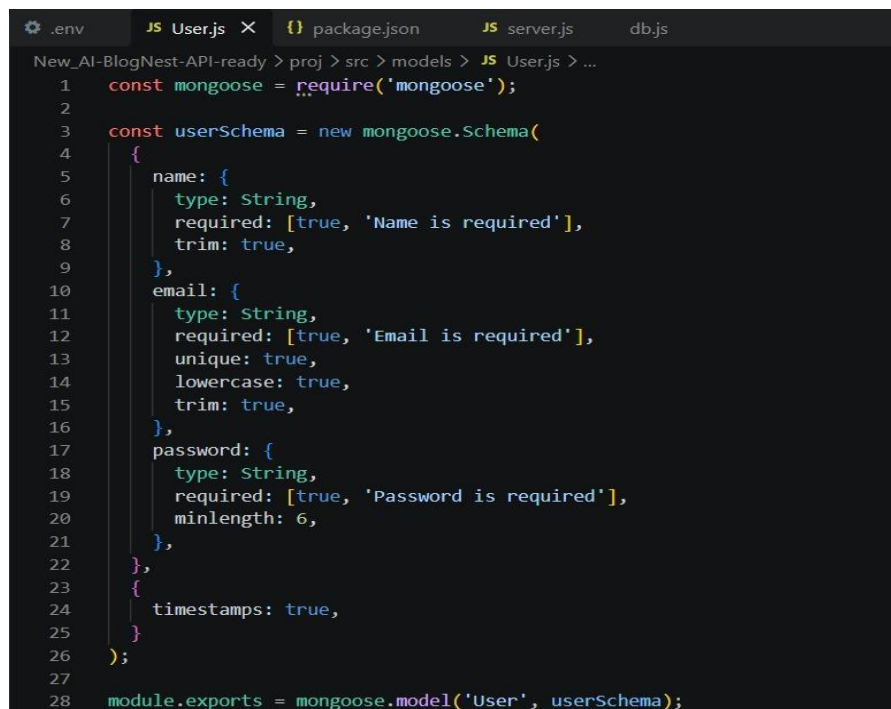
1. User Document Schema (schemas/userModel.js)

- Defines the structure of the user data stored in MongoDB and applies validation rules to incoming user records.
- It ensures that required fields such as name, email, and password are properly defined before storing user information in the database.

Mongoose Schema Definitions

1. Identity Resource Document Schema (schemas/userModel.js)

User.js

A screenshot of a code editor with a dark theme. The editor shows a file named 'User.js' with the following code:

```
1 const mongoose = require('mongoose');
2
3 const userSchema = new mongoose.Schema(
4 {
5   name: {
6     type: String,
7     required: [true, 'Name is required'],
8     trim: true,
9   },
10  email: {
11    type: String,
12    required: [true, 'Email is required'],
13    unique: true,
14    lowercase: true,
15    trim: true,
16  },
17  password: {
18    type: String,
19    required: [true, 'Password is required'],
20    minlength: 6,
21  },
22 },
23 {
24   timestamps: true,
25 }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
```

11. RESTful API Endpoints

AI Blog Nest follows RESTful API design and returns structured JSON responses for integration with frontend applications or API clients.

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate a user
GET	/api/blog	Fetch blogs
POST	/api/blog	Create a blog
DELETE	/api/blog/:id	Delete a blog
POST	/api/ai/generate-blog	Generate AI blog content
POST	/api/ai/summarize	Summarize blog content

HTTP Methods

- POST – Create new data or trigger an operation.
- GET – Retrieve data.
- PUT – Update existing data.
- DELETE – Remove data.

Epic 5: Project Execution

Step 1 – Install Dependencies

```
npm install
```

Step 2 – Configure MongoDB

- Create or select a MongoDB Atlas.
- Copy the connection string.
- Place the connection string in the MONGO_URI environment variable.

Step 3 – Start the Backend Server

- npm start
- npm install
- dir
- dir -Directory
- cd proj
- dir
- npm install
- npm audit fix
- npm install --save-dev nodemon

or, when configured with nodemon

```
npm run dev
```

Step 4 – Test the API

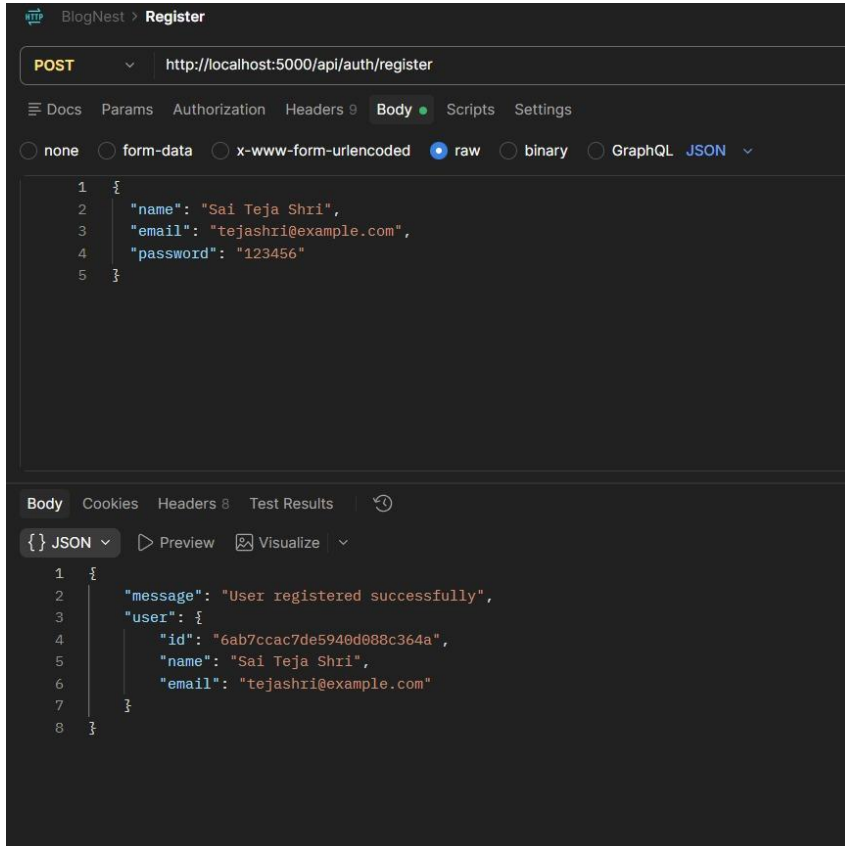
Open Postman and send requests to the configured local server.

Base URL:

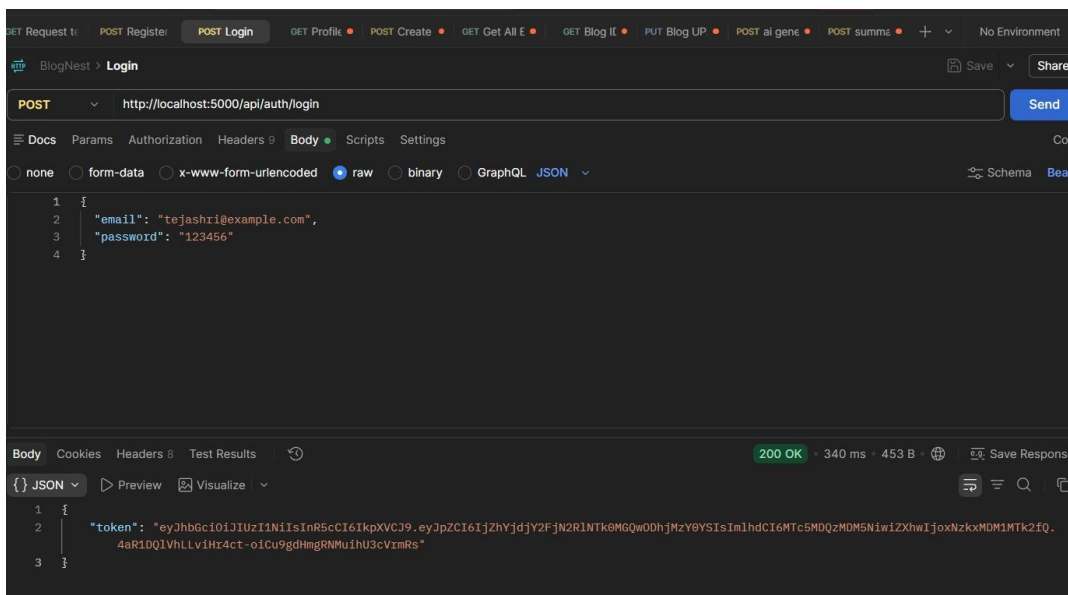
http://localhost:5000

Expected Testing Sequence

Register



Login



Create blog

The screenshot shows the Postman interface for a POST request to `http://localhost:5000/api/blogs`. The request body is a JSON object with the following structure:

```
1 {
2   "title": "The Art ",
3   "content": "Learning is a continuous journey. Reading, practicing, and staying curious can help us develop new skills and understand the
4     world better.",
5   "category": "Education"
6 }
```

The response is a 201 Created status with a response time of 111 ms and a body size of 653 B. The response body is a JSON object with the following structure:

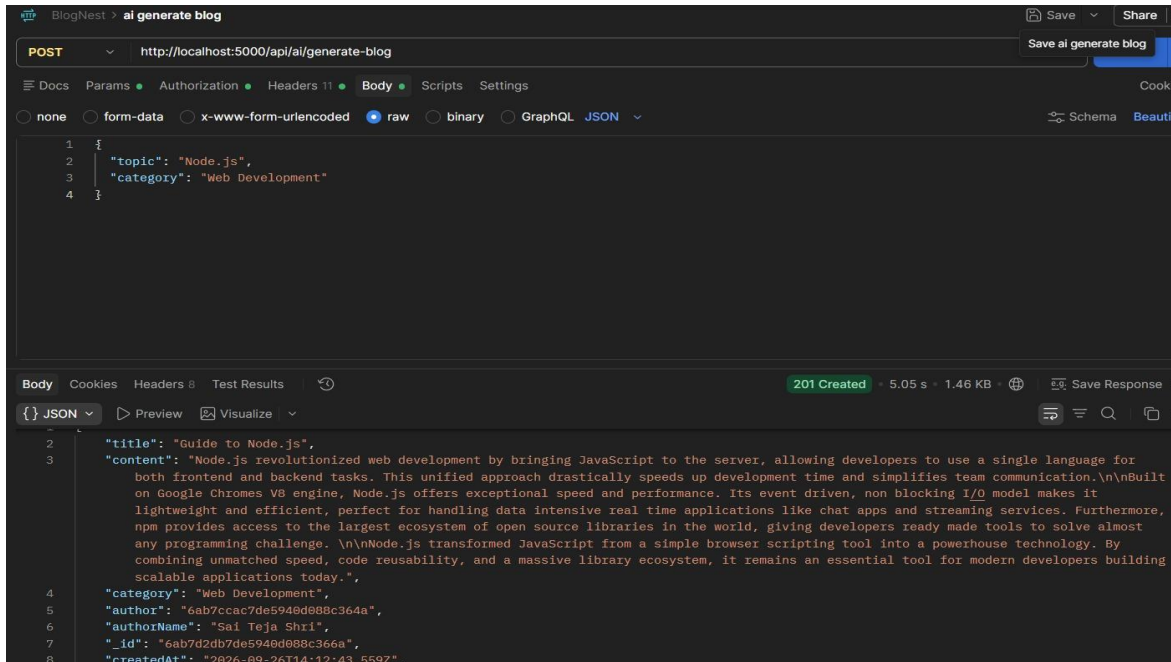
```
1 {
2   "title": "The Art",
3   "content": "Learning is a continuous journey. Reading, practicing, and staying curious can help us develop new skills and understand the
4     world better.",
5   "category": "Education",
6   "author": "6ab7ccac7de5940d088c364a",
7   "authorName": "Sai Teja Shri",
8   "_id": "6ab7cd1b7de5940d088c364f",
9   "createdAt": "2026-09-26T13:48:11.259Z",
10  "updatedAt": "2026-09-26T13:48:11.259Z",
11  "__v": 0
12 }
```

Fetch Blogs

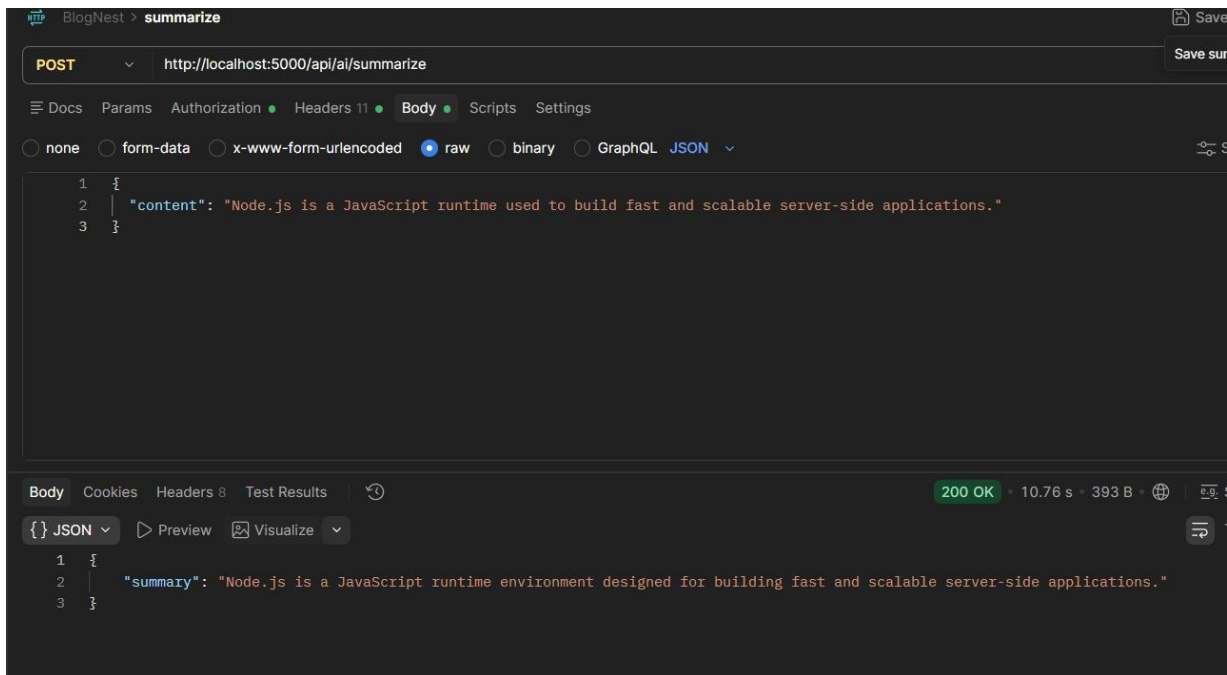
The screenshot shows the Postman interface for a GET request to `http://localhost:5000/api/blogs`. The response is a 200 OK status with a response time of 190 ms and a body size of 2.45 KB. The response body is a JSON array of blog objects. The first object in the array is:

```
3 {
4   "_id": "6ab7cd1b7de5940d088c364f",
5   "title": "The Art",
6   "content": "Learning is a continuous journey. Reading, practicing, and staying curious can help us develop new skills and understand the
7     world better.",
8   "category": "Education",
9   "author": {
10    "_id": "6ab7ccac7de5940d088c364a",
11    "name": "Sai Teja Shri",
12    "email": "tejashri@example.com"
13  },
14   "authorName": "Sai Teja Shri",
15   "createdAt": "2026-09-26T13:48:11.259Z",
16   "updatedAt": "2026-09-26T13:48:11.259Z",
17   "__v": 0
18 }
```

Generate AI blogs



Summarize Blogs



Demo Video : [Demo Videos](#)

API Testing : [API testing Video](#)

Code Drive: [Code](#)

GitHub Link: [GitHub](#)